Reviewer Pack — Minimal Rank of Tropicalized Lie Algebras Bounds DPLL Tree D…

Entry a725207fa9c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 18:19:17 UTC

Minimal Rank of Tropicalized Lie Algebras Bounds DPLL Tree Depth

Entry ID: `a725207fa9c4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 18:19:17 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Lie Algebras) **Field B** (complexity object): Complexity Theory: DPLL Search Tree

Statement:

> ['For any 3-CNF formula, the minimal rank of its associated tropicalized Lie algebra is at most a constant factor of the depth of the DPLL search tree for that formula.', 'This conjecture holds if and only if the minimal rank of the tropicalized Lie algebra corresponding to an instance with DPLL depth d is O(log d).', 'The constant factor in the above inequality is independent of the specific 3-CNF formula.']

Rationale (proposer's reasoning):

> ['Tropicalization is a tool that can map complex algebraic structures into more tractable settings, which may reveal hidden complexity-theoretic properties.', 'Lie algebras encode structural information about polynomial equations, and their tropicalized forms could potentially expose the complexity of satisfiability problems.', 'If confirmed, this would provide a novel perspective on the structure of DPLL search trees and their relation to combinatorial structures in algebra.']

Taxonomy category: `TROPICAL_LIE_ALGEBRAS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `77ad79ec0b6170e1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated 3-CNF formulas, the minimal rank of the tropicalized Lie algebra is less than or equal to a constant factor (C) times the DPLL tree depth and there exists a C such that the ratio of minimal rank to DPLL tree depth is O(log d). The conjecture is falsified if any formula has a minimal rank greater than C times its DPLL tree depth for any constant C.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND Lie algebras AND DPLL tree depth` - `DPLL search trees AND tropicalization AND minimal rank` - `complexity theory AND DPLL AND tropicalized Lie algebra`

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1105.4284v3] Lie algebras with given properties of subalgebras and elements - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/2605.05991v1] A Case-Driven Multi-Agent Framework for E-Commerce Search Relevance - [http://arxiv.org/abs/2404.08121v2] The Tropical Variety of Symmetric Rank 2 Matrices - [http://arxiv.org/abs/cs/0209032v3] Complexity Results on DPLL and Resolution - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for i in range(m):
        if rank == n:
            break
        pivot_row = i
        while pivot_row < m and A[pivot_row][i] == 0:
            pivot_row += 1
        if pivot_row == m:
            continue
        A[i], A[pivot_row] = A[pivot_row], A[i]
        for j in range(m):
            if j != i:
                factor = -A[j][i] / A[i][i]
                for k in range(n):
                    if k < i:
                        A[j][k] += factor * A[i][k]
```

```
                    elif k > i:
                        A[j][k] -= factor * A[i][k]
        rank += 1
    return rank

def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def rank(matrix):
    return gaussian_elimination(matrix)

def create_tropical_matrix(clauses, variables):
    m = len(clauses)
    n = len(variables)
    matrix = [[0 for _ in range(n)] for _ in range(m)]
    for i, clause in enumerate(clauses):
        for literal in clause:
            if literal[0] == 'x':
                var_index = int(literal[1:]) - 1
                matrix[i][var_index] = max(matrix[i][var_index], literal[2:])
            elif literal[0] == '~':
                var_index = int(literal[1:]) - 1
                matrix[i][var_index] = max(matrix[i][var_index], literal[3:])
    return matrix

def dpll_depth(clauses, variables):
    def solve(clauses, assignment):
        if not clauses:
            return True
        clause = next((c for c in clauses if any(l in assignment for l in c)), None)
        if not clause:
            return False
        literal = next(l for l in clause if l in assignment)
        if literal[0] == '~':
            literal = literal[1:]
            negated = True
```

```
        else:
            negated = False
        var_index = int(literal[1:]) - 1
        if assignment[var_index] != negated:
            return solve(clauses, assignment)
        assignment[var_index] = not negated
        if solve(clauses, assignment):
            return True
        assignment[var_index] = negated
        return False
    return len(solve(clauses, [False] * len(variables)))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n, 2 * n)
    variables = {f'x{i+1}': i for i in range(n)}
    clauses = []
    for _ in range(m):
        clause = []
        num_literals = random.randint(1, n)
        for _ in range(num_literals):
            literal = random.choice(['x', '~']) + str(random.randint(1, n)) + str(random.randint(0
            if literal not in clause:
                clause.append(literal)
        clauses.append(clause)
    tropical_matrix = create_tropical_matrix(clauses, variables)
    dpll_depth_value = dpll_depth(clauses, variables)
    rank_value = rank(tropical_matrix)
    metric_value = rank_value / math.log(dpll_depth_value + 1) if dpll_depth_value > 0 else float(
    instances_tested = 1
    conjecture_holds = metric_value <= 2.0
    counterexample = "" if conjecture_holds else f"rank={rank_value}, depth={dpll_depth_value}"
    return {
        "metric_name": "Rank/Log(DPLL Depth)",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **result}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
```

```
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"rank/log(depth) > 2\" first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_01ffa64c.py", line 137, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_01ffa64c.py", line 118, in run_trial
    tropical_matrix = create_tropical_matrix(clauses, variables)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_01ffa64c.py", line 75, in create_tropic
    matrix[i][var_index] = max(matrix[i][var_index], literal[2:])
                           ~~~~~~~~~~^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Re-run the test ensuring it completes successfully and provides results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11615
2	preregistration	ollama_remote	glm4:latest	0	0	6138
3	novelty	ollama_remote	glm4:latest	0	0	4609
4	novelty	ollama_remote	glm4:latest	0	0	6100
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16444
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11673
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12452
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16264
9	judge	ollama_remote	glm4:latest	0	0	8757

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94052 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a725207fa9c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a725207fa9c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a725207fa9c4.tar.gz` (if generated)